Test de stress de `mod_ep`

Sébastien Lefebvre

Novembre 2026 – versions 1.0.1 et 1.0.2 du module

1 Objet et méthode

Le test de charge (rapport *Test de charge*) mesurait chaque page pour un seul utilisateur. Celui-ci pousse le serveur jusqu'à la saturation, puis le maintient en charge pendant une demi-heure, et lance en même temps les traitements les plus lourds du module. Il répond à trois questions : où se situe la limite d'un serveur, que se passe-t-il au-delà, et le module y est-il pour quelque chose ?

Serveur. Une seule machine de 4 cœurs et 15 Go de mémoire, configurée comme un petit serveur de production : nginx, PHP-FPM 8.3 (16 processus, OPcache), PostgreSQL 16 réglé (`shared_buffers` 2 Go), Moodle 4.5. Le générateur de charge tourne sur la même machine et y prend environ 17 % du processeur : les débits mesurés sont donc un peu inférieurs à ceux d'un serveur dédié.

Données. Six promotions (une activité EP chacune, 200 EP au catalogue) : 10 000 étudiants, 150 enseignants communs à toutes les promotions, une synthèse enseignant liée aux six activités ; 538 000 crédits et 475 000 décisions en base (`charge_setup.php`, `synthese_setup.php`).

Charge. Des utilisateurs virtuels (`stress.py`) – 70 % d'étudiants, 22 % d'enseignants, 8 % de membres de la DEVE, répartis sur les six promotions – ouvrent leurs pages en boucle **sans temps de réflexion** : chacun produit la charge maximale possible. Un utilisateur réel lit sa page une dizaine de secondes ; un utilisateur virtuel équivaut donc à plusieurs dizaines de personnes connectées. Les étudiants s'inscrivent aussi à des EP (5 % de leurs requêtes : écritures sous verrou). Paliers de 25 à 400 utilisateurs, 60 s chacun.

2 Montée en charge

	Avant optimisation			Après optimisation			
Utilisateurs	**pages/s**	**médiane**	**99 %**	**pages/s**	**médiane**	**99 %**	**erreurs**
25	37,3	0,65 s	1,6 s	47,4	0,52 s	0,95 s	0
50	36,5	1,35 s	2,5 s	45,6	1,09 s	1,7 s	0
100	37,2	2,73 s	4,7 s	46,8	2,16 s	3,0 s	0
200	37,9	5,58 s	7,2 s	49,7	4,23 s	5,0 s	0
300	42,1	7,92 s	11,0 s	51,2	6,36 s	9,6 s	0
400	44,1	10,5 s	12,0 s	53,1	8,45 s	11,3 s	0

— **Aucune erreur** à aucun palier : ni erreur PHP ou de base de données, ni requête perdue, ni délai dépassé (60 s), y compris à 400 utilisateurs simultanés. Les inscriptions concurrentes ne produisent aucun doublon.

— **La saturation est atteinte dès 25 utilisateurs** sans temps de réflexion : le processeur est à 100 %. Au-delà, le débit reste stable (il augmente même un peu) et les requêtes attendent leur tour : le temps de réponse croît proportionnellement au nombre d'utilisateurs, sans

effondrement ni erreur. C'est le comportement attendu d'un serveur correctement borné (nombre fixe de processus PHP, file d'attente de nginx).
— La mémoire reste modeste : 2,9 Go au plus pour toute la machine à 400 utilisateurs.

3 Où va le temps

Un profil par échantillonnage (Excimer, `profil.py`) montre ce que coûte une page :

Page (après optimisation)	**ms CPU**	**module**	**base**	**rendu**	**cœur Moodle**
Tableau de bord de l'étudiant	73	3 %	15 %	26 %	56 %
Catalogue (étudiant)	58	4 %	12 %	23 %	61 %
Validation (enseignant)	61	3 %	11 %	21 %	64 %
Participants d'un EP	61	7 %	10 %	25 %	58 %
Synthèse : à traiter (6 promotions)	72	4 %	15 %	21 %	60 %
Synthèse : validation (6 promotions)	113	17 %	19 %	18 %	46 %
Pilotage (DEVE)	60	7 %	13 %	21 %	59 %
Référence : profil utilisateur de Moodle	*41*	–	7 %	33 %	59 %

Une page du module coûte de 50 à 75 ms de processeur, contre 41 ms pour une page ordinaire de Moodle. Le code du module n'en représente que quelques pour cent : l'essentiel est le socle de Moodle (thème, navigation, gabarits, chaînes de langue, connexion à la base), payé par toute page. La limite d'environ 50 pages par seconde sur 4 cœurs est donc celle de Moodle sur ce matériel, et non celle du module.

4 Optimisations

4.1 Dans le module (version 1.0.1)

Constat	**Correction**
Places prises et états par EP : 10 à 17 ms par page de catalogue, parcours de toutes les demandes de la promotion.	Index (promotion, EP, état) : la requête se lit sur l'index seul, 2,6 ms.
File de la DEVE : 19 ms pour trier les 7 000 demandes en attente avant d'en afficher 40 ; total calculé avec une jointure inutile, 17 ms.	Index (promotion, état, date) : la page se lit dans l'ordre de l'index, 0,3 ms ; total sans jointure, 1,5 ms.
Synthèse, liste des demandes de l'enseignant sur six promotions : 143 ms, dont 39 ms à mettre en forme le type et le nom de l'étudiant de chacune des milliers de lignes.	Libellés calculés une fois par type ; noms des étudiants calculés seulement pour un tri par nom ou pour les lignes affichées : 113 ms.
Participants d'un EP : toute la promotion (2 000 inscrits) chargée à chaque affichage pour proposer l'ajout d'étudiants.	Candidats lus en base avec la recherche et une limite : 83 à 61 ms.

Constat	Correction
Libellés d'état et d'année d'étude recalculés à chaque ligne (sept chaînes par ligne pour l'année).	Calculés une fois par requête.

4.2 Dans le déploiement

— **Connexions persistantes à la base** (`$CFG->dboptions['dbpersist'] = 1`, ou un pool de connexions de type pgbouncer) : la connexion à PostgreSQL coûtait environ 9 % du processeur de chaque page ; à 100 utilisateurs, le débit passe de 37,6 à 46,1 pages par seconde (+23 %). C'est le réglage le plus rentable.
— **Redis** pour les sessions et le cache applicatif : sans effet mesurable sur un serveur unique (le cache sur disque local est aussi rapide) ; il devient nécessaire avec plusieurs serveurs web.
— **Processeurs** : le débit croît avec le nombre de cœurs, la base n'étant pas le goulot (15 % du temps au plus).

Les chiffres « après optimisation » du tableau de montée en charge combinent les optimisations du module et les connexions persistantes : +27 à +35 % de débit, temps de réponse réduits d'environ 20 %.

5 Optimisations supplémentaires (version 1.0.2)

Une seconde série d'optimisations, chacune mesurée seule, dans l'ordre où elle a été ajoutée, avec un protocole court (paliers de 50, 100 et 200 utilisateurs, 60 s chacun) et un profil CPU. La mesure de référence a été refaite après un redémarrage de la machine.

Étape (cumulée)	50	100	200	gain
Référence (version 1.0.1, connexions persistantes)	44,4	47,2	46,3	–
Optimisations du module (ci-dessous)	48,2	49,2	48,7	+5 à +9 %
APCu comme cache applicatif de Moodle	53,5	55,5	57,7	+11 à +18 %
JIT de PHP 8.3 (`opcache.jit=tracing`)	56,5	60,2	61,9	+5 à +8 %
OPcache sans vérification des fichiers	60,7	59,2	62,0	≈ +2 %

Pages servies par seconde, selon le nombre d'utilisateurs simultanés.

5.1 Dans le module

— **Synthèse enseignant, liste « Validation »** : une seule requête couvre toutes les promotions, chacune avec ses propres droits ; la base trie et pagine, et seules les 40 lignes affichées sont mises en forme. 101 → 87 ms (143 ms avant la version 1.0.1).
— **Synthèse, promotions actives** : les six activités liées sont lues en une requête ; module de cours et contexte viennent des informations de cours déjà en cache de Moodle. 7,6 → 3 ms, sans mise en cache des droits (le retrait d'un droit reste immédiat).
— **Pages secondaires** : la description de l'activité n'est plus mise en forme pour être aussitôt masquée ; 4 à 6 ms de moins par page.
— **Inscription directe depuis le catalogue** : l'année d'étude se choisit sur la ligne de l'EP, sans page de confirmation. Une ruée de 300 étudiants qui s'inscrivent au même instant (`ruee.py`) est servie en 11,6 s au lieu de 15,3 s (médiane par étudiant 9,2 s au lieu de 13,0 s ; 900 pages au lieu de 1 200 ; 300 inscriptions réussies dans les deux cas).

5.2 Dans le déploiement

— **APCu** garde le cache applicatif de Moodle (chaînes de langue, configuration, informations de cours) en mémoire partagée au lieu de le relire sur disque à chaque page : c'est le gain le plus net de cette série. Le fichier de langue du module (près de 1 000 chaînes) en profite directement.
— **JIT** : le rendu des gabarits et le code de Moodle, très consommateurs de processeur, s'exécutent plus vite.
— **OPcache sans vérification des fichiers** : gain faible, dans le bruit de mesure.
— Avec APCu ou sans vérification des fichiers, il faut redémarrer PHP-FPM après chaque mise à jour du code ou mise à niveau faite en ligne de commande.
— *Ajout ultérieur.* Le test des interactions à 1 000 étudiants a montré qu'avec APCu, juste après un redémarrage de PHP-FPM ou une purge des caches, des connexions simultanées échouent sur le verrou du cache des activités des cours (`core/coursemodinfo`), qu'APCu ne sait pas tenir : ce cache doit rester sur le magasin fichier (rapport *Test des interactions à 1 000 étudiants*).

5.3 Résultat

Utilisateurs	Premier test	Version 1.0.1	Version 1.0.2	médiane 1.0.2	erreurs
25	37,3	47,4	58,9	0,42 s	0
50	36,5	45,6	59,8	0,83 s	0
100	37,2	46,8	60,6	1,67 s	0
200	37,9	49,7	59,8	3,48 s	0
300	42,1	51,2	61,1	5,24 s	0
400	44,1	53,1	60,9	7,33 s	0

Pages par seconde. Le débit d'un serveur de 4 cœurs passe d'environ 40 à 60 pages par seconde (+40 à +60 % depuis le premier test), et le temps de réponse médian diminue d'autant. Une endurance de 10 minutes à 150 utilisateurs avec tous ces réglages tient 54 à 56 pages par seconde, sans erreur et sans dérive de la mémoire.

6 Endurance

Trente minutes à 150 utilisateurs simultanés sans temps de réflexion, au-delà de la saturation (`stress.py endurance`), soit environ 80 000 pages servies :

Fenêtre	pages/s	médiane	95 %	99 %	erreurs	mémoire
0–5 min	44,4	3,33 s	3,67 s	4,17 s	0	2 638 Mo
5–10 min	44,2	3,33 s	3,85 s	4,49 s	0	2 673 Mo
10–15 min	43,5	3,41 s	3,81 s	4,21 s	0	2 748 Mo
15–20 min	45,3	3,26 s	3,70 s	4,15 s	0	2 776 Mo
20–25 min	46,5	3,18 s	3,57 s	3,96 s	0	2 780 Mo
25–30 min	43,1	3,44 s	3,80 s	4,26 s	0	2 778 Mo

Aucune dérive : débit et temps de réponse constants, mémoire stabilisée après dix minutes (les caches se remplissent puis cessent de croître), 17 connexions à la base (les 16 processus PHP et la surveillance), aucune erreur ni verrou restant.

7 Traitements lourds simultanés

Pendant que 50 utilisateurs ouvrent leurs pages en boucle (`lourds.py`), sont lancés au même instant :

Traitement	Durée	Résultat
Restauration d'une archive de 60 000 crédits dans une activité vide (ligne de commande)	20 s	59 550 crédits, 327 Mo
Export CSV détaillé de deux promotions (48 000 lignes chacune)	18 et 22 s	7,8 Mo chacun
Livret PDF de deux promotions (200 EP, livret conservé effacé)	41 et 48 s	1,3 Mo chacun
Export complet de trois promotions par trois membres de la DEVE	61 à 74 s	3,9 à 8,3 Mo
Émission groupée de 300 suppléments au diplôme (tâche ad hoc)	113 s	300 émis, 157 Mo

— Tous les traitements aboutissent, sans erreur, alors que le processeur est déjà saturé par la charge interactive : leurs durées sont celles d'un serveur occupé (l'export complet prend 10 s sur un serveur au repos).
— La mémoire reste bornée : 310 Mo au plus pour un processus PHP, 2,3 Go pour l'ensemble des processus PHP, 3,2 Go pour la machine.
— Les utilisateurs interactifs ne voient aucune erreur ; leurs temps de réponse s'allongent modérément (95 % des pages en moins de 2,1 s au lieu de 1,4 s, débit de 38,8 au lieu de 45,6 pages par seconde).
— L'émission groupée tourne en tâche de fond à environ 4 suppléments par seconde : une promotion de 1 600 étudiants demande moins de 7 minutes.

8 Capacité

Un serveur de 4 cœurs réglé comme recommandé sert environ 60 pages par seconde. Un utilisateur réel ouvre une page toutes les 10 à 15 secondes : le serveur suffit pour 600 à 900 personnes actives *au même moment*, ce qui couvre largement l'usage courant d'une école (quelques dizaines de personnes connectées en même temps).

Le seul moment exigeant est l'**ouverture des inscriptions** au catalogue, si toute une promotion se connecte à la même minute. Le test de ruée (rapport *Test de concurrence*) et les paliers ci-dessus montrent que le serveur ne commet alors aucune erreur et ne surréserve aucune place : il fait patienter. Avec l'inscription directe de la version 1.0.2, 300 étudiants qui s'inscrivent au même instant sont tous inscrits en 12 s ; pour 1 000, comptez une quarantaine de secondes au pire, et 8 cœurs divisent ce délai par deux. Étaler l'ouverture des inscriptions par promotion (périodes d'inscription des EP) est l'autre réponse, sans matériel supplémentaire.

9 Conclusion

Sous un stress bien au-delà de l'usage réel – jusqu'à 400 utilisateurs sans temps de réflexion, une demi-heure en continu, et les traitements les plus lourds lancés ensemble – le module ne commet aucune erreur, ne perd aucune requête, ne dérive pas en mémoire et garde des données cohérentes. Au-delà de la saturation, le serveur fait patienter au lieu de céder.

La limite est celle du socle Moodle sur le matériel ; le module n'y prend que quelques pour cent. Les optimisations du module (versions 1.0.1 et 1.0.2 : index, requêtes uniques paginées par la base, calculs une seule fois, inscription directe) et du déploiement (connexions persistantes, APCu, JIT) ont porté le débit d'un serveur de 4 cœurs d'environ 40 à 60 pages par seconde (+40 à +60 %), avec des temps de réponse réduits d'autant.

Les scripts sont dans `docs/audit/banc/` : `stress.py`, `profil.py`, `lourds.py`, `ruee.py`, `charge_setup.php` et `synthese_setup.php`.